

Reservation v2 API

Developer integration guide — the intelligent covers-based table booking endpoints.

Magskr backend · router prefix `/reservations/v2` · August 2026

Basics

Base URL	<code>https://magskr.com</code>
Auth	Bearer JWT in <code>Authorization: Bearer <token></code> . Two endpoints are public: availability and guest create .
Content-Type	<code>application/json</code> for all bodies.
Times	Naive Berlin wall-clock. Send <code>reserved_for</code> as ISO (e.g. <code>2026-08-13T19:00:00</code>). A trailing <code>Z</code> /offset is converted to Berlin.
Isolation	Completely separate from legacy <code>/reservations</code> and <code>/bookings</code> . Own table, own data.

Enable per store first. Until an owner sets `enabled=true` and a `max_covers` via `PUT /config`, availability returns `reason:"reservations_disabled"` and no bookings can be made.

Core concepts

- **Covers** = guests the room seats at once = capacity (`max_covers` , owner-set). A booking consumes `party_size` covers for its duration window.
- **Slot** = a bookable start time, generated from the store's opening hours by `slot_interval_minutes`. A slot is `bookable` when free covers $\geq$ party size and it isn't in the past/lead window.
- **Duration** = `default_duration_minutes`, overridable per booking (big parties $\rightarrow$ longer). It sets `reserved_until = reserved_for + duration`.
- **Status** = `pending` $\rightarrow$ `booked` (accept) / `cancelled` (decline). Auto-confirms on create if the store's `auto_accept_reservations` is on.

Typical flow

Customer booking

1. `GET /availability` with date + party size $\rightarrow$ show the bookable times.
2. Customer picks a time $\rightarrow$ `POST /guest` (or `POST /` if logged in) with that `reserved_for`.
3. On success $\rightarrow$ `201` with the reservation. On a full slot $\rightarrow$ `409` with `alternatives`; show those and retry.

Owner dashboard

- Configure once: `PUT /config`. List the day: `GET /store/today` or `POST /store/filter`.
- Accept: `PUT /{id}` `{"status":"booked"}` · Decline: `{"status":"cancelled"}` · Extend a big party: `{"duration_minutes":180}`.

Configuration

GET **/reservations/v2/config**

store owner / admin

Read a store's booking settings. Returns unsaved defaults if none exist yet.

```
curl 'https://magskr.com/reservations/v2/config?store_id=13' \
-H 'Authorization: Bearer <owner-token>'
```

```
{
  "store_id": 13,
  "enabled": true,
  "max_covers": 40,
  "slot_interval_minutes": 30,
  "default_duration_minutes": 45,
  "max_party_size": 10,
  "lead_time_minutes": 30,
  "booking_window_days": 30
}
```

PUT **/reservations/v2/config**

store owner / admin

`store_id` is a query param; the body holds any subset of fields to change (upsert).

```
curl -X PUT 'https://magskr.com/reservations/v2/config?store_id=13' \
-H 'Authorization: Bearer <owner-token>' \
-H 'Content-Type: application/json' \
-d '{ "enabled": true, "max_covers": 40, "slot_interval_minutes": 30,
      "default_duration_minutes": 45, "max_party_size": 10 }'
```

Field	Type	Meaning
enabled	bool	Master switch for v2 booking at this store
max_covers	int	Capacity — guests seatable at once
slot_interval_minutes	int	Spacing between slot start times (default 30)
default_duration_minutes	int	Default table hold (default 45)
max_party_size	int	Largest online booking; bigger → "please call" (default 10)
lead_time_minutes	int	No booking within X min of now (default 30)
booking_window_days	int	How far ahead bookings open (default 30)

Availability

The "is it free / show other times" call. Query: `store_id`, `date` (YYYY-MM-DD), `party_size` (default 2).

```
curl 'https://magskr.com/reservations/v2/availability?store_id=13&date=2026-08-13&party_size=4'
```

```
{
  "store_id": 13, "date": "2026-08-13", "party_size": 4,
  "enabled": true, "reason": null,
  "slots": [
    { "time": "18:30", "datetime": "2026-08-13T18:30:00", "available": 16, "total": 40,
      "bookable": true },
    { "time": "19:00", "datetime": "2026-08-13T19:00:00", "available": 2, "total": 40,
      "bookable": false },
    { "time": "19:30", "datetime": "2026-08-13T19:30:00", "available": 0, "total": 40,
      "bookable": false },
    { "time": "20:00", "datetime": "2026-08-13T20:00:00", "available": 30, "total": 40,
      "bookable": true }
  ]
}
```

When `slots` is empty, `reason` explains why: `reservations_disabled`, `not_configured`, `closed` (no hours that day), `closed_holiday`, `outside_booking_window`, `party_too_large`, `invalid_party_size`. Show only `bookable:true` slots as selectable; render the rest greyed/"full".

Create a reservation

POST /reservations/v2/guest

public · 10/min

POST /reservations/v2/

logged-in customer

Same body for both. The authenticated version fills name/email from the account when omitted. The server re-checks availability under a per-store lock, so two people can't grab the last covers.

```
curl -X POST 'https://magskr.com/reservations/v2/guest' \
-H 'Content-Type: application/json' \
-d '{ "store_id": 13, "party_size": 4,
      "reserved_for": "2026-08-13T20:00:00",
      "customer_name": "Weber", "customer_phone": "+49 170 1234567",
      "customer_email": "weber@example.com", "note": "window seat please" }'
```

Field	Req	Notes
store_id	yes	
party_size	yes	Covers consumed
reserved_for	yes	Must equal a bookable slot's <code>datetime</code>
duration_minutes	no	Defaults to the store's <code>default_duration_minutes</code>
customer_name / _phone / _email	no*	*email needed to send confirmation
note	no	

201 Created — the reservation:

```
{
  "id": 5012, "store_id": 13, "user_id": null,
  "party_size": 4,
  "reserved_for": "2026-08-13T20:00:00",
  "reserved_until": "2026-08-13T20:45:00",
  "duration_minutes": 45,
  "status": "pending",
  "table_number": null,
  "customer_name": "Weber", "customer_phone": "+49 170 1234567",
  "customer_email": "weber@example.com", "note": "window seat please",
  "created_at": "2026-08-06T13:20:11"
}
```

`status` is `booked` immediately if the store has `auto_accept_reservations` on; otherwise `pending` until the owner accepts.

Error responses

HTTP	detail.reason	Meaning
403	—	Reservations not enabled for this store
422	party_too_large	Over <code>max_party_size</code> — also returns <code>max_party_size</code>
422	invalid_slot	Time isn't a bookable slot — returns <code>alternatives</code>
409	slot_full	No covers left — returns <code>available</code> + <code>alternatives</code>
409	closed / closed_holiday / outside_booking_window	Day/time not open for booking

```
// 409 slot_full
{ "detail": {
  "reason": "slot_full", "available": 2,
  "alternatives": [
    { "time": "20:00", "datetime": "2026-08-13T20:00:00", "available": 30, "total": 40,
    "bookable": true },
    { "time": "20:30", "datetime": "2026-08-13T20:30:00", "available": 30, "total": 40,
    "bookable": true }
  ] } }
```

Manage reservations (staff)

PUT

/reservations/v2/{id}

staff · owning customer (cancel)

Accept, decline, or edit. Send any subset. Staff-only fields: `party_size` , `reserved_for` , `duration_minutes` , contact fields, `isActive` . A logged-in customer may only set `status:"cancelled"` on their own. `customer_message` is a one-shot line added to the status-change email (not stored).

```
# accept
curl -X PUT 'https://magskr.com/reservations/v2/5012' -H 'Authorization: Bearer <owner>' \
  -H 'Content-Type: application/json' -d '{"status":"booked"}'

# decline with a note to the guest
curl -X PUT '.../reservations/v2/5012' -H 'Authorization: Bearer <owner>' \
  -H 'Content-Type: application/json' \
  -d '{"status":"cancelled","customer_message":"Sorry, fully booked that evening."}'

# extend a big party to 3 hours (reshapes availability for everyone)
curl -X PUT '.../reservations/v2/5012' -H 'Authorization: Bearer <owner>' \
  -H 'Content-Type: application/json' -d '{"duration_minutes":180}'
```

Returns the updated reservation. Fires the status email, customer push, and the dashboard Pusher event.

GET

/reservations/v2/store/today

staff

Today's reservations for a store, by `reserved_for` . Query `store_id` (owners scoped automatically).

POST

/reservations/v2/store/filter

staff

```
curl -X POST '.../reservations/v2/store/filter' -H 'Authorization: Bearer <owner>' \
  -H 'Content-Type: application/json' \
  -d '{ "store_id": 13, "target_date": "2026-08-13", "limit": 100, "offset": 0 }'
```

GET

/reservations/v2/

staff

GET

/reservations/v2/my

customer

GET

/reservations/v2/{id}

staff / owning customer

DELETE

/reservations/v2/{id}

store owner / admin

GET / and GET /my accept an optional `?store_id` filter. DELETE hard-deletes; prefer PUT `{"status":"cancelled"}` or `{"isActive":false}` to keep the record.

Field reference — reservation object

Field	Type	Notes
id	int	
store_id	int	

user_id	int?	null for guest bookings
party_size	int	covers
reserved_for / reserved_until	datetime	window start / end
duration_minutes	int	editable per booking
status	str	pending booked cancelled
table_number	int?	reserved for future floor-plan (Approach B); unused now
customer_name / _phone / _email, note	str?	
created_at	datetime	

Companion docs in `docs/table/` : `EXPLAINER.pdf` (concepts + diagrams) and `IMPLEMENTATION.md` (build/architecture).